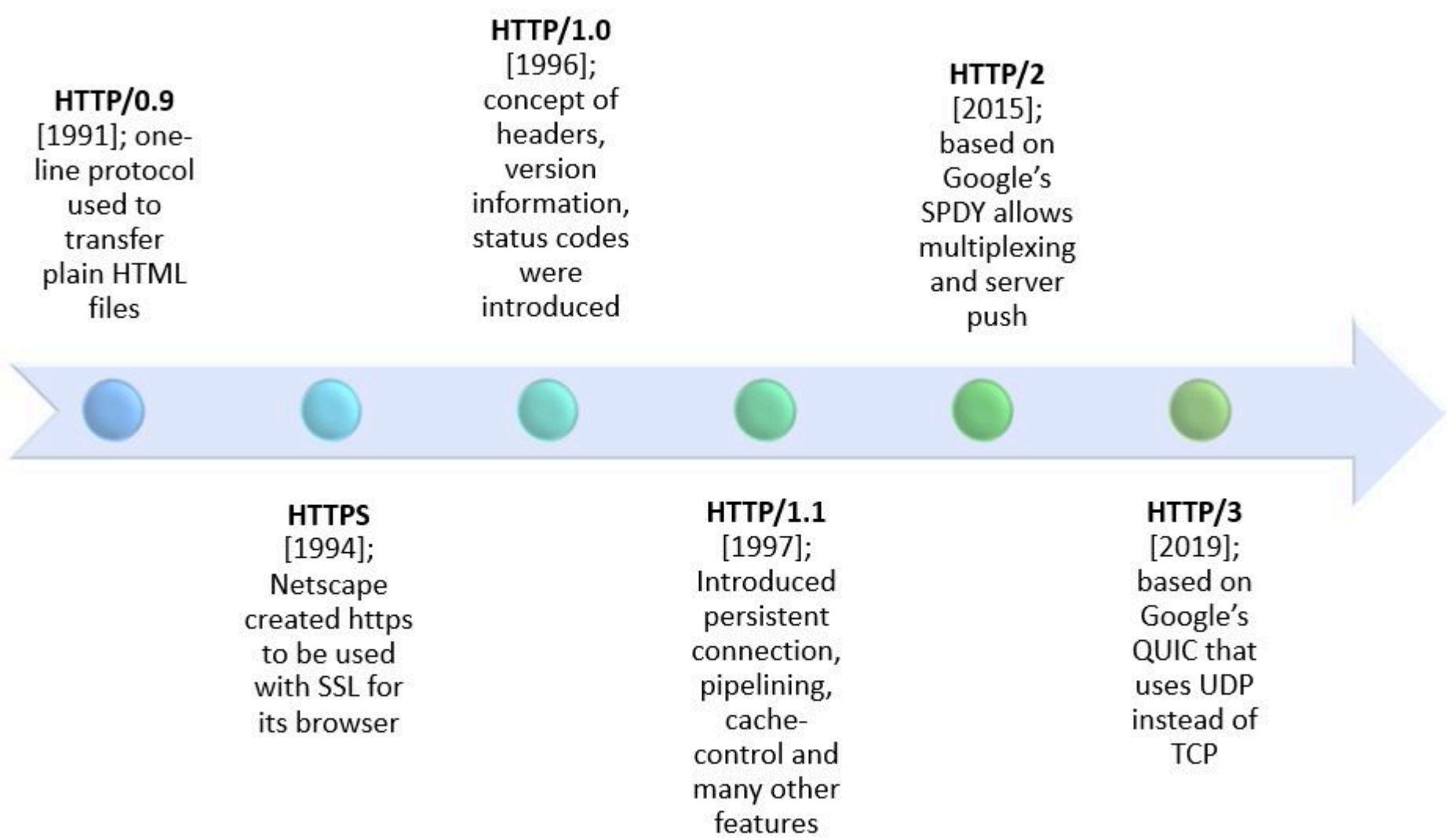
6. HTTP and HTTPS

HTTP – What, Why, and How it is Stateless

- HTTP can be understood using a real-world analogy where a customer walks into a shop, requests a product, receives it, and leaves, and when they return again, the shopkeeper has no memory of the previous interaction.
  - Every visit is treated as a completely new interaction
  - No past data or history is remembered
- HTTP (Hyper Text Transfer Protocol) is a communication protocol used between a client (browser/application) and a server.
  - Communication happens in a simple flow:
    - Client sends a request
    - Server sends a response
  - After the response:
    - Connection is completed
    - No memory of interaction is stored
- The most important property of HTTP is that it is **stateless**.
  - Server does not store:
    - Previous requests
    - User history
    - Interaction context
  - Each request is:
    - Independent
    - Self-contained
- This stateless design was intentional in the early web.
  - Web was mainly used for:
    - Serving static documents
  - No need for:
    - Continuous sessions
    - User tracking
- Statelessness helps in making systems scalable and efficient.
  - Server does not need to:
    - Store client data
    - Manage session memory
  - Reduces:
    - Memory usage
    - System complexity
- Because HTTP is stateless, every request must include all required information.
  - Examples:
    - Authentication details
    - Resource identifiers (URL)
    - Required headers
- A major advantage of statelessness is **horizontal scalability**.
  - Any server can handle any request
  - No dependency on previous interactions
  - Works well in distributed systems and load-balanced environments

HTTP Versions – Evolution from 1.0 to 3



HTTP 1.0

- HTTP 1.0 was the earliest widely used version of HTTP.
  - Each request required:
    - Opening a new TCP connection
    - Receiving the response
    - Closing the connection immediately
  - This caused performance issues:
    - Opening/closing connections repeatedly is expensive
    - Introduces high latency
  - Example problem:
    - A webpage with multiple resources (images, CSS, JS)
    - Browser had to create a **separate connection for each**
    - Result → very slow page load times

HTTP 1.1

- HTTP 1.1 improved performance by optimizing connection usage.
  - Introduced **persistent connections (keep-alive)**:
    - Same TCP connection reused for multiple requests
    - Reduced connection overhead
  - Introduced **pipelining**:
    - Multiple requests sent without waiting for responses
    - Improved throughput
  - Limitation:
    - **Head-of-Line (HOL) blocking**
      - Responses must come back in order
      - One slow request delays all others

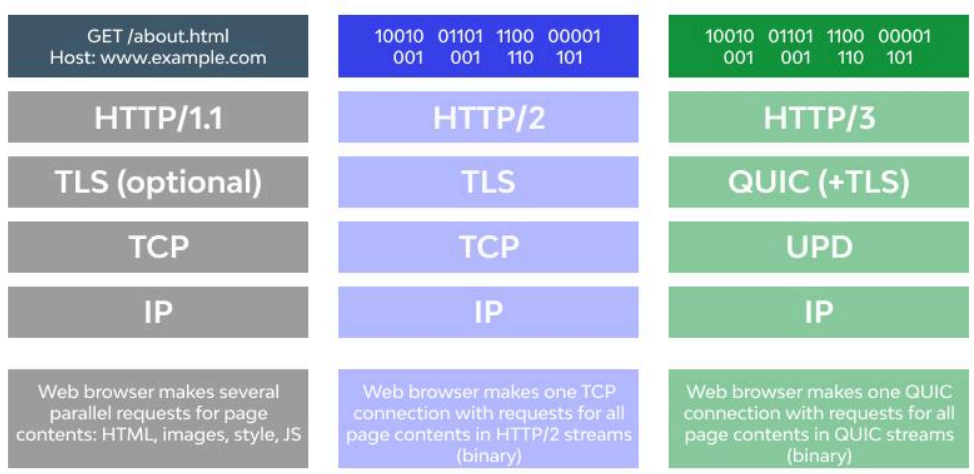
HTTP 2

- HTTP 2 brought major improvements in how data is transmitted.
  - Introduced **multiplexing**:
    - Multiple requests and responses handled simultaneously
    - All over a single TCP connection

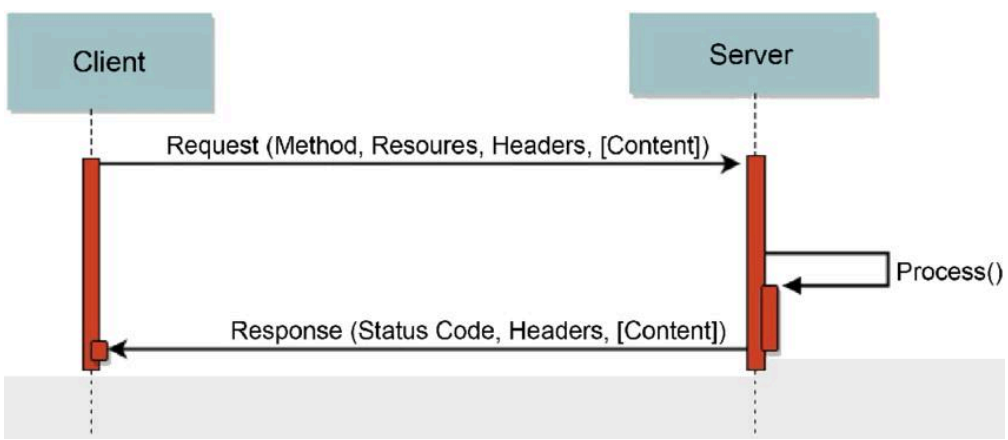
- Uses **binary framing**:
  - Requests split into smaller frames
  - Frames can be interleaved and processed independently
- Benefits:
  - Eliminates HOL blocking at HTTP layer
  - Faster and more efficient communication
- Introduced **header compression (HPACK)**:
  - Reduces repeated header data
  - Saves bandwidth
- Introduced **server push**:
  - Server can send resources before client requests them
  - Improves page load performance

HTTP 3

- HTTP 3 changes the transport layer by replacing TCP.
  - Uses **QUIC protocol (built on UDP)**:
    - Handles reliability and ordering itself
  - Problem in HTTP 2:
    - TCP still had Head-of-Line blocking
    - Lost packet blocks all streams
  - QUIC solution:
    - Uses **independent streams**
    - Packet loss in one stream does not affect others
  - Benefits:
    - Better performance on slow/unreliable networks
    - Reduced latency
  - Faster connection setup:
    - Combines transport and TLS handshake
    - Fewer round trips needed



HTTP Request and Response Structure



HTTP Request

- An HTTP request is sent by the client to the server to ask for a resource or perform an action.
  - **Request Line**:
    - First line of the request
    - Contains:

        - HTTP method (GET, POST, etc.)
        - Resource path (URL)
        - HTTP version
    - Example:
        - `GET /products/123 HTTP/1.1`
- **Headers**:
    - Key-value pairs that provide metadata about the request
    - Help the server understand how to process it
    - Common headers:
        - `Host` → specifies target domain
        - `Accept` → tells server expected response format
        - `Authorization` → carries authentication token
- **Request Body**:
    - Optional part of the request
    - Used mainly in:
        - POST
        - PUT
        - PATCH
    - Contains data sent to the server (usually JSON)
- **Example Understanding**:
    - A real request includes:
        - Request line + headers + optional body
    - Authorization header often contains:
        - Bearer token for authentication

HTTP Response

- An HTTP response is sent by the server back to the client after processing the request.
    - **Status Line**:
        - First line of the response
        - Contains:
            - HTTP version
            - Status code
            - Status message
        - Example:
            - `HTTP/1.1 200 OK`
    - **Headers**:
        - Provide metadata about the response
        - Help the client understand how to handle it
        - Common headers:
            - `Content-Type` → format of response data
            - `Content-Length` → size of response
            - `Cache-Control` → caching rules
    - **Response Body**:
        - Contains the actual data returned by the server
        - Can be:
            - JSON
            - HTML
            - Images
            - Other formats
    - **Example Understanding**:
        - A typical response may:
            - Return JSON data (like product details)
            - Include headers to guide parsing and caching

HTTP Methods

- HTTP methods define the intended action of a request and specify what operation the client wants the server to perform on a resource.

- **Idempotent** :- means that making the same request multiple times has the same effect on the server as making it once.

GET

- Used to retrieve data from the server.
  - Does **not modify** any data on the server.
  - Considered **safe** (no side effects).
  - Considered **idempotent**:
    - Multiple identical requests return the same result.

POST

- Used to create a new resource on the server.
  - Sends data in the request body.
  - **Modifies server state**.
  - **Not idempotent**:
    - Repeating the request can create multiple resources.

PUT

- Used to completely replace an existing resource with new data.
  - Sends full resource data in the request body.
  - Overwrites the existing resource.
  - **Idempotent**:
    - Repeating the same request results in the same final state.

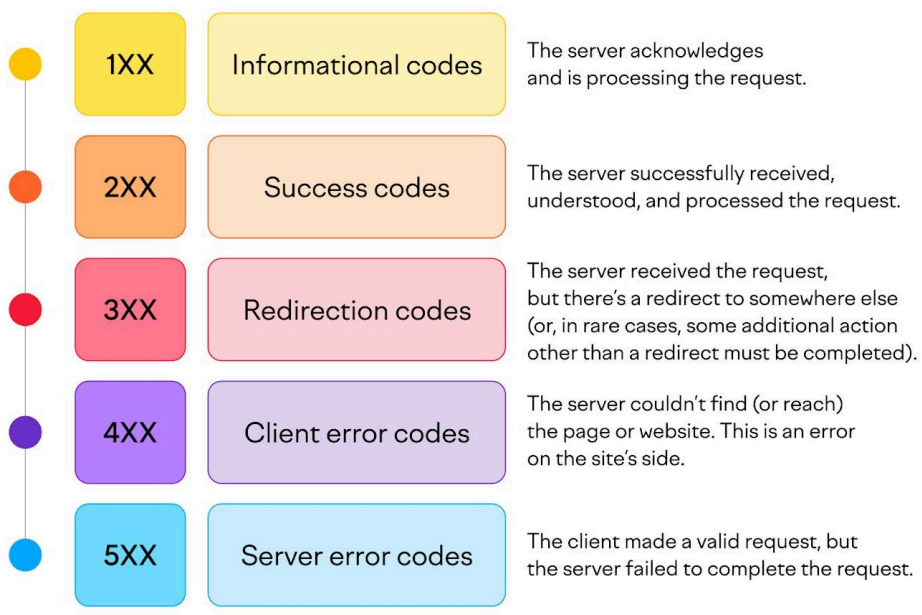
PATCH

- Used to partially update an existing resource.
  - Sends only the fields that need to be updated.
  - More efficient than PUT when updating small parts of data.
  - May or may not be idempotent depending on implementation.

DELETE

- Used to remove a resource from the server.
  - Deletes the specified resource.
  - **Idempotent**:
    - Deleting the same resource multiple times results in the same outcome.

HTTP Status Codes



Here it is converted into **clean, structured points with subpoints for each HTTP status code category**, keeping it simple and interview-friendly:

1xx – Informational (Rarely used in practice)

- These codes indicate that the request has been received and the process is continuing.

2xx – Success

- These status codes indicate that the request was successfully received, understood, and processed by the server.

- **200 OK**
  - The request was successful
  - Response contains the requested data
- **201 Created**
  - A new resource was successfully created
  - Commonly used after POST requests
- **204 No Content**
  - Request was successful but no data is returned
  - Common after DELETE or update operations

3xx – Redirection

- These codes indicate that further action needs to be taken by the client to complete the request.
  - **301 Moved Permanently**
    - Resource has been permanently moved to a new URL
    - Future requests should use the new URL
  - **302 Found**
    - Resource is temporarily available at a different URL
    - Original URL should still be used later
  - **304 Not Modified**
    - Resource has not changed since last request
    - Client should use cached version

4xx – Client Errors

- These codes indicate that there was an error in the request made by the client.
  - **400 Bad Request**
    - Request is malformed or invalid
  - **401 Unauthorized**
    - Authentication is required or failed
  - **403 Forbidden**
    - Client is authenticated but does not have permission
  - **404 Not Found**
    - Requested resource does not exist
  - **429 Too Many Requests**
    - Client has sent too many requests (rate limiting)

5xx – Server Errors

- These codes indicate that the server failed to fulfill a valid request due to internal issues.
  - **500 Internal Server Error**
    - Generic server failure
  - **502 Bad Gateway**
    - Server received an invalid response from an upstream server
  - **503 Service Unavailable**
    - Server is overloaded or temporarily down
  - **504 Gateway Timeout**
    - Upstream server did not respond in time

Headers in Depth

- Headers are the metadata of HTTP that travel with every request and response, carrying important information that defines how communication between the client and server happens.
- They help in controlling:
  - Data format
  - Authentication
  - Caching
  - Security rules

Host

- The **Host** header tells the server which domain is being requested.
  - Important because:
    - One server can host multiple websites

- Helps the server:
  - Route the request to the correct application
- Example:
  - `Host: api.example.com`

Content-Type

- The **Content-Type** header tells the receiver the format of the body.
  - Helps in:
    - Understanding how to parse the data
  - Common values:
    - `application/json` → body is JSON
    - `multipart/form-data` → used for file uploads

Accept

- The **Accept** header is sent by the client to tell the server what format it can handle.
  - Used for:
    - Content negotiation
  - Example:
    - `Accept: application/json` → client expects JSON
  - Server action:
    - Matches this and sets response `Content-Type`

Authorization

- The **Authorization** header carries authentication credentials.
  - Common method:
    - Bearer token
  - Example:
    - `Authorization: Bearer eyJhbGciOiJIUzI1NiJ9 ...`
  - Server role:
    - Validates the token
    - Allows or rejects the request

Cache-Control

- The **Cache-Control** header defines how caching should behave.
  - Common directives:
    - `max-age=3600` → cache for 3600 seconds
    - `no-cache` → must revalidate before using
    - `no-store` → do not cache at all
  - Applies to:
    - Browser
    - CDN
    - Intermediate caches

ETag

- The **ETag** header is a unique identifier for a specific version of a resource.
  - Used for:
    - Cache validation
  - Example:
    - `ETag: "abc123def"`
  - Client behavior:
    - Stores it for future checks

If-None-Match

- The **If-None-Match** header is used by the client to send the stored ETag back to the server.
  - Purpose:
    - Check if resource has changed
  - Flow:
    - Client sends:
      - `If-None-Match: "abc123def"`

        - Server compares with current ETag
    - Result:
        - Match → `304 Not Modified` (no body)
        - No match → new response sent

Set-Cookie

- The **Set-Cookie** header is used by the server to send cookies to the client.
    - Example:
        - `Set-Cookie: sessionId=xyz; HttpOnly; Secure; SameSite=Strict`
    - Client behavior:
        - Stores cookie
        - Sends it back in future requests using `Cookie` header

CORS Headers

- CORS headers control whether cross-origin requests are allowed.
    - Example:
        - `Access-Control-Allow-Origin`
    - Purpose:
        - Tells the browser if another domain is allowed to access this resource
    - Important:
        - Enforced by browser for security

HTTPS

- HTTP sends everything as plain text over the wire
- Anyone between you and the server (ISP, someone on same WiFi, a router) can read every byte
- HTTPS solves this by wrapping HTTP inside an encrypted tunnel
- That tunnel is built by a protocol called TLS
- HTTPS = HTTP + TLS, nothing more

SSL vs TLS — Clearing the Confusion

- SSL (Secure Sockets Layer) was the original protocol built by Netscape in the 1990s
- SSL 1.0 was never released, SSL 2.0 and 3.0 had serious security vulnerabilities
- TLS (Transport Layer Security) was introduced as the successor to SSL
    - TLS 1.0 → 1.1 → 1.2 → 1.3 (current standard)
- SSL is technically dead and deprecated
- But the industry still says "SSL certificate", "SSL handshake" colloquially — they mean TLS
- When someone says SSL today, mentally replace it with TLS

TLS versions and why they matter

- TLS 1.2 — widely used, still secure, but has some older cipher support which is a weakness
- TLS 1.3 — current standard, released in 2018
    - Removed old, weak encryption algorithms entirely
    - Reduced handshake from 2 round trips to 1 round trip (faster connection setup)
    - Better forward secrecy by default (explained below)

Certificates — The Foundation of Trust

- Before encryption can happen, the client needs to know it is talking to the real server and not an imposter
- This is solved by SSL/TLS certificates
- A certificate contains:
    - The domain name it is issued for (e.g. bank.com)
    - The server's public key
    - The issuer — which Certificate Authority (CA) signed it
    - Expiry date
    - A digital signature from the CA

Certificate Authority (CA)

- A CA is a globally trusted third party — examples are DigiCert, Let's Encrypt, GlobalSign
- Your browser and OS come preloaded with a list of trusted CAs
- When a server presents its certificate, the browser checks:

    - Was this certificate signed by a CA I trust?
    - Is the domain in the certificate the same as the domain I'm visiting?
    - Is the certificate expired?
- If all three pass → trust is established
- If any one fails → browser shows the red "Your connection is not private" warning

Asymmetric vs Symmetric Encryption

- **Asymmetric encryption** uses two keys — a public key and a private key
    - What the public key encrypts, only the private key can decrypt
    - The public key is shared openly, private key is kept secret on the server
    - It is computationally expensive and slow
    - Used during the handshake phase only
- **Symmetric encryption** uses one shared secret key on both sides
    - Same key encrypts and decrypts
    - It is very fast
    - Used for encrypting actual data after the handshake
- The whole point of the TLS handshake is to use asymmetric encryption to safely agree on a shared symmetric key, then switch to symmetric for the rest of the session

The TLS Handshake — Step by Step

Let's use TLS 1.3 since it is the current standard

Step 1 — Client Hello

- You type https://bank.com and your browser initiates a connection on port 443
- Browser sends a Client Hello message to the server containing:
    - TLS version it supports (e.g. TLS 1.3)
    - A list of cipher suites it supports (encryption algorithms it knows how to use)
    - A randomly generated number called client random
    - A key share — the client's part of the Diffie-Hellman key exchange (more on this below)

Step 2 — Server Hello

- Server responds with a Server Hello containing:
    - The TLS version and cipher suite it picked from the client's list
    - A randomly generated number called server random
    - Its own key share — the server's part of the Diffie-Hellman exchange
    - Its SSL certificate (containing the server's public key, signed by a CA)

Step 3 — Certificate Verification

- Browser receives the certificate and verifies it
    - Checks the CA signature — is this signed by someone I trust?
    - Checks the domain — does the certificate say bank.com?
    - Checks expiry — is it still valid?
- If verification passes, the browser trusts that it is talking to the real bank.com

Step 4 — Key Exchange using Diffie-Hellman

- This is the clever part — both sides arrive at the same secret key without ever sending the key over the wire
- How it works conceptually:
    - Both client and server agree on public parameters (think of them as a shared base)
    - Client generates a private value, server generates a private value — neither shares these
    - They exchange their computed public values (key shares sent in step 1 and 2)
    - Using their own private value + the other side's public value, both independently compute the same final shared secret
    - An attacker watching the network sees the public values but cannot derive the shared secret without the private values
- This shared secret is then used to derive the actual symmetric session keys for encrypting data

Step 5 — Finished Messages

- Both client and server send a Finished message
- This message is encrypted using the newly derived session keys
- It contains a hash of the entire handshake so both sides can verify nothing was tampered with during the handshake
- If both sides can decrypt and verify each other's Finished message → handshake is complete

Step 6 — Encrypted Communication Begins

- All HTTP data from this point is encrypted using the symmetric session keys
- Nobody in the middle can read anything

Forward Secrecy — Why It Matters

- Old versions of TLS used RSA key exchange — the shared secret was encrypted with the server's public key
- Problem: if an attacker recorded all encrypted traffic and later stole the server's private key, they could decrypt all past sessions retroactively
- TLS 1.3 enforces Diffie-Hellman by default which gives Forward Secrecy
- Forward Secrecy means: each session uses a fresh, temporary key pair
- Even if the server's private key is compromised later, past sessions cannot be decrypted because those temporary keys no longer exist

Complete HTTPS Flow End to End

- User types https://bank.com
- DNS resolves bank.com to an IP address
- Browser opens a TCP connection to that IP on port 443
- TLS handshake happens (as above) — encrypted tunnel is established
- Browser sends the HTTP request through the encrypted tunnel
  - GET /dashboard HTTP/1.1, headers, cookies — all encrypted
- Server decrypts the request, processes it, sends back HTTP response — encrypted
- Browser decrypts the response and renders the page
- The user sees their bank dashboard — nobody in the middle saw anything

Key Takeaways

- HTTPS is not a new protocol — it is HTTP inside a TLS tunnel
- SSL is dead, TLS is what actually runs today
- Certificates prove identity, Diffie-Hellman establishes a shared secret, symmetric encryption does the actual data encryption
- The handshake in TLS 1.3 takes only 1 round trip, making it fast
- Forward secrecy ensures that compromising the server today does not expose past conversations